

6

Java I/O: Context-Specific Deserialization Filters

This chapter includes 13 problems related to Java serialization/deserialization processes. We start with classical problems like serializing/deserializing objects to `byte[]`, `String`, and XML. We continue with JDK 9 deserialization filters meant to prevent deserialization vulnerabilities, and we finish with JDK 17 (JEP 415, final) context-specific deserialization filters.

At the end of this chapter, you'll be skilled in solving almost any problem related to serializing/deserializing objects in Java.

Problems

Use the following problems to test your programming prowess on Java serialization/deserialization. I strongly encourage you to give each problem a try before you turn to the solutions and download the example programs:

131. **Serializing objects to byte arrays:** Write a Java application that exposes two helper methods for serializing/deserializing objects to/from `byte[]`.
132. **Serializing objects to strings:** Write a Java application that exposes two helper methods for serializing/deserializing objects to/from `String`.
133. **Serializing objects to XML:** Exemplify at least two approaches for serializing/deserializing objects to/from XML format.
134. **Introducing JDK 9 deserialization filters:** Provide a brief introduction to JDK 9 deserialization filters including some insights into the `ObjectInputFilter` API.
135. **Implementing a custom pattern-based `ObjectInputFilter`:** Provide an example of implementing and setting a custom pattern-based filter via the `ObjectInputFilter` API.
136. **Implementing a custom class `ObjectInputFilter`:** Exemplify the creation of an `ObjectInputFilter` via a class implementation.

- 137. **Implementing a custom method `ObjectInputFilter`:** Exemplify the creation of an `ObjectInputFilter` via a method implementation.
- 138. **Implementing a custom lambda `ObjectInputFilter`:** Exemplify the creation of an `ObjectInputFilter` via a lambda expression.
- 139. **Avoiding `StackOverflowError` at deserialization:** First, write a snippet of code that can be successfully serialized but, in the deserialization phase, causes a `StackOverflowError`. Second, write a filter that avoids this unpleasant scenario.
- 140. **Avoiding DoS attacks at deserialization:** First, write a snippet of code that can be successfully serialized but, in the deserialization phase, causes a DoS attack. Second, write a filter that avoids this unpleasant scenario.
- 141. **Introducing JDK 17 easy filter creation:** Explain and exemplify the usage of the JDK 17 `allowFilter()` and `rejectFilter()` methods.
- 142. **Tackling context-specific deserialization filters:** Explain and exemplify the usage of JDK 17 Filter Factories.
- 143. **Monitoring deserialization via JFR:** Exemplify the usage of **Java Flight Recorder (JFR)** for monitoring a deserialization event.

The following sections describe solutions to the preceding problems. Remember that there usually isn't a single correct way to solve a particular problem. Also, remember that the explanations shown here include only the most interesting and important details needed to solve the problems. Download the example solutions to see additional details and to experiment with the programs at

<https://github.com/PacktPublishing/Java-Coding-Problems-Second-Edition/tree/main/Chapter06>.

131. Serializing objects to byte arrays

In *Chapter 4, Problem 94*, we talked about the serialization/deserialization of Java records, so you should be pretty familiar with these operations. In a nutshell, serialization is the process of transforming an in-memory object into a stream of bytes that can also be stored in memory or written to a file, network, database, external storage, and so on. Deserialization is the reverse process, that is, recreating the object state in memory from the given stream of bytes.

A Java object is serializable if its class implements `java.io.Serializable` (or, `java.io.Externalizable`). Accomplishing serialization/deserialization takes place via the `java.io.ObjectOutputStream` and `java.io.ObjectInputStream` classes and `writeObject()` / `readObject()` methods.

For instance, let's assume the following `Melon` class:

```
public class Melon implements Serializable {  
    private final String type;  
    private final float weight;  
    // constructor, getters  
}
```

And, an instance of `Melon`:

```
Melon melon = new Melon("Gac", 2500);
```

Serializing the `melon` instance into a byte array can be accomplished as follows:

```
public static byte[] objectToBytes(Serializable obj)  
    throws IOException {  
  
    try (ByteArrayOutputStream baos  
        = new ByteArrayOutputStream();  
        ObjectOutputStream ois  
        = new ObjectOutputStream(baos)) {  
        ois.writeObject(obj);  
  
        return baos.toByteArray();  
    }  
}
```

Of course, we can use this helper to serialize any other object, but for the `melon` instance, we call it as follows:

```
byte[] melonSer = Converters.objectToBytes(melon);
```

Deserialization is done via another helper that uses `readObject()` as follows:

```
public static Object bytesToObject(byte[] bytes)  
    throws IOException, ClassNotFoundException {  
    try (InputStream is = new ByteArrayInputStream(bytes);  
        ObjectInputStream ois = new ObjectInputStream(is)) {  
        return ois.readObject();  
    }  
}
```

```
}
}
```

We can use this helper to deserialize any other object from a byte array, but for the `melonSer`, we call it as follows:

```
Melon melonDeser = (Melon) Converters.bytesToObject(melonSer);
```

The returned `melonDeser` restores the initial object state even if it is not the same instance. In the bundled code, you can also see an approach based on Apache Commons Lang.

132. Serializing objects to strings

In the previous problem, you saw how to serialize objects to byte arrays. If we work a little bit on a byte array, we can obtain a string representation of serialization. For instance, we can rely on `java.util.Base64` to encode a byte array to `String` as follows:

```
public static String objectToString(Serializable obj) throws IOException {
    try ( ByteArrayOutputStream baos = new ByteArrayOutputStream();
          ObjectOutputStream ois = new ObjectOutputStream(baos)) {
        ois.writeObject(obj);

        return Base64.getEncoder().encodeToString(baos.toByteArray());
    }
}
```

A possible output looks like this:

```
r00ABXNyABZtb2R1cm4uY2hhbGxlbmd1Lk1lbG9u2WrnGA2MxZ4CAAJGAAZ3ZWlnaHRMAAR0eXB1dAAST GphdmEvbGFuZy9TdHJpbmc7eHBFHEAAAdAADR2Fj
```

And, the code to obtain such a `String` is as follows:

```
String melonSer = Converters.objectToString(melon);
```

The reverse process relies on the `Base64` decoder as follows:


```
public static Object stringToObject(String obj)
    throws IOException, ClassNotFoundException {
    byte[] data = Base64.getDecoder().decode(obj);
    try ( ObjectInputStream ois = new ObjectInputStream(
        new ByteArrayInputStream(data))) {
        return ois.readObject();
    }
}
```

Calling this method is straightforward:

```
Melon melonDeser = (Melon)
    Converters.stringToObject(melonSer);
```

The `melonDeser` object is the result of deserializing the previous string.

133. Serializing objects to XML

Serializing/deserializing objects to XML via the JDK API can be accomplished via `java.beans.XMLEncoder`, respectively `XMLDecoder`. The `XMLEncoder` API relies on Java Reflection to discover the object's fields and write them in XML format.

This class can encode objects that respect the Java Beans contract

(<https://docs.oracle.com/javase/tutorial/javabeans/writing/index.html>).

Basically, the object's class should contain a public no-arguments constructor and public getters and setters for `private` / `protected` fields/properties.

Implementing `Serializable` is not mandatory for `XMLEncoder` / `XMLDecoder`, so we can serialize/deserialize objects that don't implement `Serializable`. Here, it is a helper method that encodes the given `Object` to XML:

```
public static String objectToXML(Object obj)
    throws IOException {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    try ( XMLEncoder encoder = new XMLEncoder(
        new BufferedOutputStream(baos))) {
        encoder.writeObject(obj);
    }
    baos.close();
    return new String(baos.toByteArray());
}
```

The reverse process (deserialization) uses the `XMLDecoder` as follows:

```
public static Object XMLToObject(String xml)
    throws IOException {
    try (InputStream is
        = new ByteArrayInputStream(xml.getBytes());
        XMLDecoder decoder = new XMLDecoder(is)) {
        return decoder.readObject();
    }
}
```

The `XMLEncoder` / `XMLDecoder` is much more flexible than the `writeObject()` / `readObject()` API. For instance, if a field/property is added/removed/renamed or its type has changed, then the decoding process skips everything it cannot decode and tries to decode as much as possible without throwing an exception.

Another common approach relies on the third-party library Jackson 2.x, which comes with `XmlMapper`. This library should be added as a dependency (of course, if you don't have it already present in your project):

```
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
  <version>2.x</version>
</dependency>
```

Next, we create an instance of `XmlMapper` :

```
XmlMapper xmlMapper = new XmlMapper();
```

Via `XmlMapper`, we can serialize objects as XML as follows (there is no problem if the object's class doesn't implement `Serializable` and/or it doesn't contain setters):

```
public static String objectToXMLJackson(Object obj)
    throws JsonProcessingException {
    XmlMapper xmlMapper = new XmlMapper();
```

```
if (xmlMapper.canSerialize(obj.getClass())) {  
    return xmlMapper.writeValueAsString(obj);  
}  
return "";  
}
```

Calling this method can be done as follows (`melon` is an instance of the `Melon` class):

```
String melonSer = Converters.objectToXMLJackson(melon);
```

The reverse process can rely on `readValue()` as follows:

```
public static <T> T XMLToObjectJackson(  
    String xml, Class<T> clazz)  
    throws JsonProcessingException {  
    XmlMapper xmlMapper = new XmlMapper();  
    return xmlMapper.readValue(xml, clazz);  
}
```

Calling this method can be done as follows:

```
Melon melonDeser = Converters  
    .XMLToObjectJackson(melonSer, Melon.class);
```

Take your time to explore the `XmlMapper` API since it has a lot more to offer. For now, consider running the bundled code to see the XML produced by each of these two approaches.

If you plan to serialize/deserialize an object to JSON, then consider *Java Coding Problems, First Edition, Problem 141*, for a comprehensive set of examples based on JSONB, Jackson, and Gson.

134. Introducing JDK 9 deserialization filters

As you know from *Chapter 4, Problem 94*, deserialization is exposed to vulnerabilities that may cause serious security issues. In other words, between serializa-

tion–deserialization cycles, an untrusted process (attacker) can modify/alter the serialization form to execute arbitrary code, sneak in malicious data, and so on.

In order to prevent such vulnerabilities, JDK 9 has introduced the possibility of creating restrictions via filters meant to accept/reject deserialization based on specific predicates. A deserialization filter intercepts a stream that expects to be deserialized and applies to it one or more predicates that should be successfully passed in order to proceed with deserialization. If a predicate fails, then deserialization doesn't even start and the stream is rejected.

There are two kinds of filters:

- **JVM-wide filters:** Filters applied to every deserialization that takes place in the JVM. The behavior of these filters is tightly coupled with how they are combined with other filters (if any).
- **Stream filters:** Filters that operate on all `ObjectInputStream` instances of an application (*stream-global filters*) or on certain `ObjectInputStream` instances (*stream-specific filters*).

We can create the following types of filters:

- Filters based on patterns (known as *pattern-based filters*): These filters can be used to filter modules, packages, or classes via string patterns. They can be applied without touching the code (as JVM-wide filters) or they can be created via the `ObjectInputFilter` API (as *pattern-based stream filters*).
- Filters based on the `ObjectInputFilter` API: This API allows us to define filters directly in code. Usually, such filters are defined based on string patterns or Java Reflection.

Pattern-based filters

Let's see several filters based on string patterns. For instance, this filter accepts all classes from the `foo` package (and from any other package that is not `buzz`) and rejects all classes from the `buzz` package (a class that passes a pattern that starts with `!` is rejected):

```
foo.*;!buzz.*
```

Patterns are delimited via semicolons (;) and white spaces are considered as part of the pattern.

The following filter rejects only the `modern.challenge.Melon` class:

```
!modern.challenge.Melon
```

The following filter rejects the `Melon` class from the `modern.challenge` package and accepts all other classes from this package (the `*` is the wildcard used to represent unspecified class/package/module names):

```
!modern.challenge.Melon;modern.challenge.*;!*
```

The following filter accepts all classes from the `foo` package and its sub-packages (notice the `**` wildcard):

```
foo.**
```

The following filter accepts all classes starting with `Hash` :

```
Hash*
```

Besides filtering classes, packages, and modules, we can also define the so-called *resource filters*, which allow us to accept/reject resources based on an object's graph complexity and size. In this context, we have `maxdepth` (the maximum graph depth), `maxarray` (the maximum array size), `maxrefs` (the maximum number of references between objects of a graph), and `maxbytes` (the maximum number of stream bytes). Here is an example:

```
maxdepth=5;maxarray=1000;maxrefs=50 foo.buzz.App
```

Now, let's see how we can use such filters.

Applying a pattern-based filter per application

If we want to apply a *pattern-based filter* to a single run of an application, then we can rely on the `jdk.serialFilter` system property. Without touching the code, we use this system property at the command line as in the following example:

```
java -Djdk.serialFilter=foo.**;Hash* foo.buzz.App
```

A system property replaces a Security Property value.

Applying a pattern-based filter to all applications in a process

To apply a *pattern-based filter* to all applications in a process, we should follow two steps (again, we don't touch the application code):

1. In an editor (for instance, Notepad or Wordpad), open the `java.security` file. In JDK 6-8, this file is located in `$JAVA_HOME/lib/security/java.security`, while in JDK 9+, it is in `$JAVA_HOME/conf/security/java.security`.
2. Edit this file by appending the pattern to the `jdk.serialFilter` Security Property.

Done!

ObjectInputFilter-based filters

Via the `ObjectInputFilter` API, we can create custom filters based on string patterns and Java Reflection. These filters can be applied to certain streams (*stream-specific filters*) or to all streams (*stream-global filters*) and can be implemented as *pattern-based filters*, as classes, methods, or lambda expressions.

First, we implement the filter via the `ObjectInputFilter` API. Second, we set the filter on all/certain `ObjectInputStream` instances. Setting the filter as a *stream-global filter* is done via

```
ObjectInputFilter.Config.setSerialFilter(ObjectInputFilter filter).
```

On the other hand, setting the filter as a *stream-specific filter* can be done via

```
ObjectInputStream.setObjectInputFilter(ObjectInputFilter filter).
```

For instance, creating a *pattern-based filter* via this API can be done by calling the `Config.createFilter(String pattern)` method.

A custom filter defined as a class is done by implementing the `ObjectInputFilter` functional interface and overriding the `Status checkInput(FilterInfo filterInfo)` method.

A custom filter defined as a method is commonly done via a static method as `static ObjectInputFilter.Status someFilter(FilterInfo info) {...}`.

And, a custom filter defined as a lambda expression is commonly expressed as `ois.setObjectInputFilter(f -> (...))`, where `f` is `ObjectInputFilter` and `ois` is an instance of `ObjectInputStream`.

A filter returns a *status* (`java.io.ObjectInputFilter.Status`), which can be `ALLOWED`, `REJECTED`, or `UNDECIDED`.

In the next problems, we will explore these statements via examples.

135. Implementing a custom pattern-based ObjectInputFilter

Let's assume that we already have the `Melon` class and the helper methods for serializing/deserializing objects to/from byte arrays from *Problem 131*.

Creating a *pattern-based filter* via the `ObjectInputFilter` API can be done by calling the `Config.createFilter(String pattern)` method. For instance, the following filter rejects the `modern.challenge.Melon` class:

```
ObjectInputFilter melonFilter = ObjectInputFilter.Config
    .createFilter("!modern.challenge.Melon;");
```

We can set this filter as a *stream-global filter* via `setSerialFilter()` as follows:

```
ObjectInputFilter.Config.setSerialFilter(melonFilter);
```

If we need to get access to a *stream-global filter*, then we can call `getSerialFilter()`:

```
ObjectInputFilter serialFilter =
```

```
ObjectInputFilter.Config.getSerialFilter();
```

Any stream deserialization in this application will pass through this filter, which will reject any instance of `modern.challenge.Melon`. You can practice this filter in the bundled code.

On the other hand, if we want to set this on a specific stream, then we can modify our `Converters.bytesToObject()` method to accept a filter as follows:

```
public static Object bytesToObject(byte[] bytes,
                                   ObjectInputFilter filter)
    throws IOException, ClassNotFoundException {
    try (InputStream is = new ByteArrayInputStream(bytes);
         ObjectInputStream ois = new ObjectInputStream(is)) {
        // set the filter
        ois.setObjectInputFilter(filter);
        return ois.readObject();
    }
}
```

If we pass `null` as `filter`, then the filter will not be applied. Otherwise, the passed filter will be applied to the current stream:

```
Melon melon = new Melon("Gac", 2500);
// serialization works as usual
byte[] melonSer = Converters.objectToBytes(melon);
// here, we pass the melonFilter, which rejects the instances
// of modern.challenge.Melon, so deserialization is rejected
Melon melonDeser = (Melon) Converters.bytesToObject(
    melonSer, melonFilter);
```

In this example, the `melonFilter` will reject deserialization and the output will be as follows:

```
Exception in thread "main" java.io.InvalidClassException: filter status: REJECTED
...
```

You can also practice this filter in the bundled code.

136. Implementing a custom class ObjectInputFilter

Let's assume that we already have the `Melon` class and the helper methods for serializing/deserializing objects to/from byte arrays from *Problem 131*.

An `ObjectInputFilter` can be written via a dedicated class by implementing the `ObjectInputFilter` functional interface as in the following example:

```
public final class MelonFilter implements ObjectInputFilter {  
    @Override  
    public Status checkInput(FilterInfo filterInfo) {  
        Class<?> clazz = filterInfo.serialClass();  
        if (clazz != null) {  
            // or, clazz.getName().equals("modern.challenge.Melon")  
            return  
                !(clazz.getPackage().getName().equals("modern.challenge")  
                  && clazz.getSimpleName().equals("Melon"))  
                ? Status.ALLOWED : Status.REJECTED;  
        }  
        return Status.UNDECIDED;  
    }  
}
```

This filter is exactly the same as the *pattern-based filter*,
`!modern.challenge.Melon`, only that it is expressed via Java Reflection.

We can set this filter as a *stream-global filter* as follows:

```
ObjectInputFilter.Config.setSerialFilter(new MelonFilter());
```

Or, as a *stream-specific filter* as follows:

```
Melon melonDeser = (Melon) Converters.bytesToObject(  
    melonSer, new MelonFilter());
```

Of course, the `bytesToObject()` accepts an argument of type
`ObjectInputFilter` and sets this filter accordingly (ois is the specific

`ObjectInputStream`):

```
ois.setObjectInputFilter(filter);
```

In this example, the `MelonFilter` will reject deserialization and the output will be as follows:

```
Exception in thread "main" java.io.InvalidClassException: filter status: REJECTED
...
```

You can practice both approaches (*stream-global* and *stream-specific*) in the bundled code.

137. Implementing a custom method `ObjectInputFilter`

Let's assume that we already have the `Melon` class and the helper methods for serializing/deserializing objects to/from byte arrays from *Problem 131*.

An `ObjectInputFilter` can be written via a dedicated method as in the following example:

```
public final class Filters {
    private Filters() {
        throw new AssertionError("Cannot be instantiated");
    }
    public static ObjectInputFilter.Status melonFilter(
        FilterInfo info) {
        Class<?> clazz = info.serialClass();
        if (clazz != null) {
            // or, clazz.getName().equals("modern.challenge.Melon")
            return
                !(clazz.getPackage().getName().equals("modern.challenge")
                    && clazz.getSimpleName().equals("Melon"))
                ? Status.ALLOWED : Status.REJECTED;
        }
        return Status.UNDECIDED;
    }
}
```

Of course, you can add more filters in this class.

We can set this filter as a *stream-global filter* as follows:

```
ObjectInputFilter.Config  
    .setSerialFilter(Filters::melonFilter);
```

Or, as a *stream-specific filter* as follows:

```
Melon melonDeser = (Melon) Converters.bytesToObject(  
    melonSer, Filters::melonFilter);
```

Of course, the `bytesToObject()` accepts an argument of type `ObjectInputFilter` and sets this filter accordingly (`ois` is the specific `ObjectInputStream`):

```
ois.setObjectInputFilter(filter);
```

In this example, the `Filters::melonFilter` will reject deserialization and the output will be as follows:

```
Exception in thread "main" java.io.InvalidClassException: filter status: REJECTED  
...
```

You can check out both approaches (*stream-global* and *stream-specific*) in the bundled code. Moreover, you can also practice another example that rejects all instances of `Melon` and its subclasses based on a *stream-global filter*.

138. Implementing a custom lambda ObjectInputFilter

Let's assume that we already have the `Melon` class and the helper methods for serializing/deserializing objects to/from byte arrays from *Problem 131*.

An `ObjectInputFilter` can be written via a dedicated lambda and set as a *stream-global filter* as follows:

```
ObjectInputFilter.Config
    .setSerialFilter(f -> ((f.serialClass() != null)
        // or, filter.serialClass().getName().equals(
        //     "modern.challenge.Melon")
        && f.serialClass().getPackage()
            .getName().equals("modern.challenge")
        && f.serialClass().getSimpleName().equals("Melon")))
    ? Status.REJECTED : Status.UNDECIDED);
```

Or, as a *stream-specific filter* as follows:

```
Melon melonDeser = (Melon) Converters.bytesToObject(melonSer,
    f -> ((f.serialClass() != null)
        // or, filter.serialClass().getName().equals(
        //     "modern.challenge.Melon")
        && f.serialClass().getPackage()
            .getName().equals("modern.challenge")
        && f.serialClass().getSimpleName().equals("Melon")))
    ? Status.REJECTED : Status.UNDECIDED);
```

You can practice these examples in the bundled code.

139. Avoiding StackOverflowError at deserialization

Let's consider the following snippet of code:

```
// 'mapOfSets' is the object to serialize/deserialize
HashMap<Set, Integer> mapOfSets = new HashMap<>();
Set<Set> set = new HashSet<>();
mapOfSets.put(set, 1);
set.add(set);
```

We plan to serialize the `mapOfSets` object as follows (I assume that `Converters.objectToBytes()` is well-known from the previous problems):

```
byte[] mapSer = Converters.objectToBytes(mapOfSets);
```

Everything works just fine until we try to deserialize `mapSer`. At that moment, instead of a valid object, we will get a `StackOverflowError` as follows:

```
Exception in thread "main" java.lang.StackOverflowError
  at java.base/java.util.HashMap$KeyIterator
    .<init>(HashMap.java:1626)
  at java.base/java.util.HashMap$KeySet
    .iterator(HashMap.java:991)
  at java.base/java.util.HashSet
    .iterator(HashSet.java:182)
  at java.base/java.util.AbstractSet
    .hashCode(AbstractSet.java:120)
  at java.base/java.util.AbstractSet
    .hashCode(AbstractSet.java:124)
  ...
```

The deserialization process got stuck in the `hashCode()` method of `Set`. The solution is to create a filter that will reject deserialization if the object has a graph depth bigger than 2. This can be a *pattern-based filter* as follows:

```
ObjectInputFilter filter = ObjectInputFilter.Config
    .createFilter("maxdepth=2;java.base/*;!*");
```

Next, call the deserialization process with this filter:

```
HashMap mapDeser = (HashMap) Converters
    .bytesToObject(mapSer, filter);
```

I assume that `Converters.bytesToObject()` is well-known from the previous problems. This time, instead of getting `StackOverflowError`, the deserialization is rejected by the filter.

140. Avoiding DoS attacks at deserialization

Denial-of-service (DoS) attacks are typically malicious actions meant to trigger, in a short period of time, a lot of requests to a server, application, and so on. Generally speaking, a DoS attack is any kind of action that intentionally/accidentally overwhelms a process and forces it to slow down or

even crash. Let's see a snippet of code that is a good candidate for representing a DoS attack in the deserialization phase:

```
ArrayList<Object> startList = new ArrayList<>();
List<Object> list1 = startList;
List<Object> list2 = new ArrayList<>();
for (int i = 0; i < 101; i++) {
    List<Object> sublist1 = new ArrayList<>();
    List<Object> sublist2 = new ArrayList<>();
    sublist1.add("value: " + i);
    list1.add(sublist1);
    list1.add(sublist2);
    list2.add(sublist1);
    list2.add(sublist2);
    list1 = sublist1;
    list2 = sublist2;
}
```

We plan to serialize the `startList` object as follows (I assume that `Converters.objectToBytes()` is well-known from the previous problems):

```
byte[] startListSer = Converters.objectToBytes(startList);
```

Everything works just fine until we try to deserialize `startListSer`. At that moment, instead of a valid object, we will get...nothing! Actually, the application starts normally, but it just hangs there in the deserialization phase. The system slows down and, after a while, it will eventually crash.

The object graph is too deep to be deserialized and that leads to a behavior similar to a DoS attack. The solution is to create a filter that will reject deserialization if the object has a graph depth bigger than a safe value. This can be a *pattern-based filter* as follows:

```
ObjectInputFilter filter = ObjectInputFilter.Config
    .createFilter("maxdepth=10;java.base/*;!*");
```

Next, call the deserialization process with this filter:

```
ArrayList startListDeser = (ArrayList)
    Converters.bytesToObject(startListSer, filter);
```

I assume that `Converters.bytesToObject()` is well-known from the previous problems. This time, the deserialization is rejected by the filter and the DoS attack is prevented.

141. Introducing JDK 17 easy filter creation

Starting with JDK 17, we can express filters more intuitively and readably via two convenient methods named `allowFilter()` and `rejectFilter()`. And, since the best way to learn is with an example, here is a usage case of these two convenient methods:

```
public final class Filters {
    private Filters() {
        throw new AssertionError("Cannot be instantiated");
    }
    public static ObjectInputFilter allowMelonFilter() {
        ObjectInputFilter filter = ObjectInputFilter.allowFilter(
            clazz -> Melon.class.isAssignableFrom(clazz),
            ObjectInputFilter.Status.REJECTED);
        return filter;
    }
    public static ObjectInputFilter rejectMuskmelonFilter() {
        ObjectInputFilter filter = ObjectInputFilter.rejectFilter(
            clazz -> Muskmelon.class.isAssignableFrom(clazz),
            ObjectInputFilter.Status.UNDECIDED);
        return filter;
    }
}
```

The `allowMelonFilter()` relies on `ObjectInputFilter.allowFilter()` to allow only objects that are instances of `Melon` or subclasses of `Melon`. The `rejectMuskmelonFilter()` relies on `ObjectInputFilter.rejectFilter()` to reject all objects that are instances of `Muskmelon` or subclasses of `Muskmelon`.

We can use each of these filters as you already know from the previous problems, so let's tackle another use case. Let's assume that we have the following hierarchy of classes:

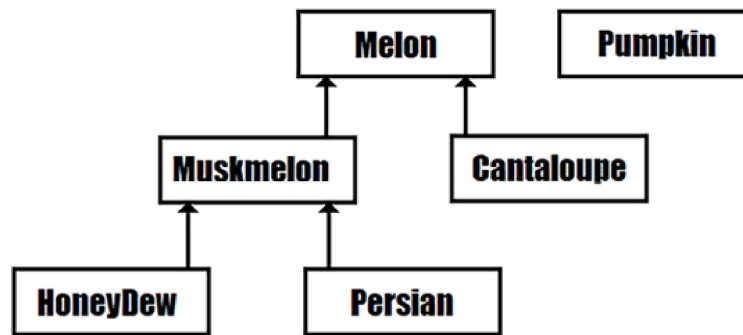


Figure 6.1: An arbitrary hierarchy of classes

Let's assume that we want to deserialize only objects that are instances of `Melon` or subclasses of `Melon` but they are not `Muskmelon` or subclasses of `Muskmelon`. In other words, we allow deserializing instances of `Melon` and `Cantaloupe`.

If we apply the `allowMelonFilter()`, then we will deserialize instances of `Melon`, `Muskmelon`, `Cantaloupe`, `HoneyDew`, and `Persian` since all these are `Melon`.

On the other hand, if we apply `rejectMuskmelonFilter()`, then we will deserialize instances of `Melon`, `Cantaloupe`, and `Pumpkin` since these are not `Muskmelon`.

But, if we apply `rejectMuskmelonFilter()` after applying `allowMelonFilter()`, then we will deserialize only `Melon` and `Cantaloupe`, which is exactly what we want.

Intuitively, we may think to chain our filters by writing something like this (`ois` is the current `ObjectInputStream`):

```
ois.setObjectInputFilter(Filters.allowMelonFilter());
ois.setObjectInputFilter(Filters.rejectMuskmelonFilter());
```

But, this will not work! It will cause a `java.lang.IllegalStateException: filter cannot be set more than once`.

The solution relies on `ObjectInputFilter.merge(filter, anotherFilter)`, which returns a filter that merges the status of these two filters by applying the following logic:

1. Call `filter` and get the returned `status`
2. If the returned `status` is `REJECTED`, then return it
3. Call `anotherFilter` and get the returned `otherStatus`
4. If `anotherStatus` is `REJECTED`, then return it
5. If either `status` or `otherStatus` is `ALLOWED`, then return `ALLOWED`
6. Otherwise, return `UNDECIDED`

Nevertheless, if `anotherFilter` is `null`, the filter is returned.

Based on this logic, we can merge the status of `Filters.allowMelonFilter()` with the status of `Filters.rejectMuskmelonFilter()` as follows:

```
public static Object bytesToObject(byte[] bytes,
    ObjectInputFilter allowFilter,
    ObjectInputFilter rejectFilter)
    throws IOException, ClassNotFoundException {
    try (InputStream is = new ByteArrayInputStream(bytes);
        ObjectInputStream ois = new ObjectInputStream(is)) {
        // set the filters
        ObjectInputFilter filters = ObjectInputFilter.merge(
            allowFilter, rejectFilter);
        ois.setObjectInputFilter(filters);
        return ois.readObject();
    }
}
```

Next, let's talk about JDK 17 Filter Factories.

142. Tackling context-specific deserialization filters

JDK 17 enriched the deserialization filter capabilities with the implementation of JEP 415, *Context-Specific Deserialization Filters*.

Practically, JDK 17 added the so-called Filter Factories. Depending on the context, a Filter Factory can dynamically decide what filters to use for a stream.

Applying a Filter Factory per application

If we want to apply a Filter Factory to a single run of an application, then we can rely on the `jdk.serialFilterFactory` system property. Without touching the code, we use this system property at the command line as in the following example:

```
java -Djdk.serialFilterFactory=FilterFactoryName YourApp
```

The `FilterFactoryName` is the fully qualified name of the Filter Factory, which is a public class that can be accessed by the application class loader, and it was set before the first deserialization.

Applying a Filter Factory to all applications in a process

To apply a Filter Factory to all applications in a process, we should follow two steps (again, we don't touch the application code):

1. In an editor (for instance, Notepad or Wordpad), open the `java.security` file. In JDK 6-8, this file is located in `$JAVA_HOME/lib/security/java.security`, while in JDK 9+, it is in `$JAVA_HOME/conf/security/java.security`.
2. Edit this file by appending the Filter Factory to the `jdk.serialFilterFactory` Security Property.

Applying a Filter Factory via `ObjectInputFilter.Config`

Alternatively, a Filter Factory can be set directly in code via `ObjectInputFilter.Config` as follows:

```
ObjectInputFilter.Config  
    .setSerialFilterFactory(FilterFactoryInstance);
```

The `FilterFactoryInstance` argument is an instance of a Filter Factory. This Filter Factory will be applied to all streams from the current application.

Implementing a Filter Factory

A Filter Factory is implemented as a `BinaryOperator<ObjectInputFilter>`. The `apply(ObjectInputFilter current, ObjectInputFilter next)` method provides the *current filter* and the *next* or *requested filter*.

In order to see how it works, let's assume that we have the following three filters:

```
public final class Filters {
    private Filters() {
        throw new AssertionError("Cannot be instantiated");
    }
    public static ObjectInputFilter allowMelonFilter() {
        ObjectInputFilter filter = ObjectInputFilter.allowFilter(
            clazz -> Melon.class.isAssignableFrom(clazz),
            ObjectInputFilter.Status.REJECTED);
        return filter;
    }
    public static ObjectInputFilter rejectMuskmelonFilter() {
        ObjectInputFilter filter = ObjectInputFilter.rejectFilter(
            clazz -> Muskmelon.class.isAssignableFrom(clazz),
            ObjectInputFilter.Status.UNDECIDED);
        return filter;
    }
    public static ObjectInputFilter packageFilter() {
        return ObjectInputFilter.Config.createFilter(
            "modern.challenge.*;!*");
    }
}
```

The `Filters.allowMelonFilter()` is set as a *stream-global filter* as follows:

```
ObjectInputFilter.Config.setSerialFilter(
    Filters.allowMelonFilter());
```

The `Filters.rejectMuskmelonFilter()` is set as a *stream-specific filter* as follows:

```
Melon melon = new Melon("Melon", 2400);
// serialization
```

```
byte[] melonSer = Converters.objectToBytes(melon);
// deserialization
Melon melonDeser = (Melon) Converters.bytesToObject(
    melonSer, Filters.rejectMuskmelonFilter());
```

And, the `Filters.packageFilter()` is set in the Filter Factory as follows:

```
public class MelonFilterFactory implements
    BinaryOperator<ObjectInputFilter> {
    @Override
    public ObjectInputFilter apply(
        ObjectInputFilter current, ObjectInputFilter next) {
        System.out.println();
        System.out.println("Current filter: " + current);
        System.out.println("Requested filter: " + next);
        if (current == null && next != null) {
            return ObjectInputFilter.merge(
                next, Filters.packageFilter());
        }
        return ObjectInputFilter.merge(next, current);
    }
}
```

The `MelonFilterFactory` is set via `ObjectInputFilter.Config` before any deserialization takes place:

```
MelonFilterFactory filterFactory = new MelonFilterFactory();
ObjectInputFilter.Config
    .setSerialFilterFactory(filterFactory);
```

Now that everything is in place, let's see what's happening. The `apply()` method is called twice. The first time it is called is when the `ObjectInputStream ois` is created, and we obtain the following output:

```
Current filter: null
Requested filter:
predicate(modern.challenge.Filters$$Lambda$4/0x000000080
1001800@ba8a1dc, ifTrue: ALLOWED,ifFalse: REJECTED)
```

The *current filter* is `null`, and the *requested filter* is `Filters.allowMelonFilter()`. Since the *current filter* is `null` and the *requested filter* is not `null`, we decided to return a filter as the result of merging the status of the *requested filter* with the status of the `Filters.packageFilter()`.

The second time the `apply()` method is called happens when the `ois.setObjectInputFilter(filter)` is called in `Converters.bytesToObject(byte[] bytes, ObjectInputFilter filter)`. We have the following output:

```
Current filter: merge(predicate(modern.challenge.Filters$$Lambda$4/0x0000000801001800@ba8a1dc, ifTrue: ALLOWED, ifFalse:REJECTED), modern.c
Requested filter: predicate(modern.challenge.Filters$$Lambda$10/0x0000000801002a10@484b61fc, ifTrue: REJECTED, ifFalse:UNDECIDED)
```

This time, the *current* and *requested filters* are not `null`, so we merge their statuses again. Finally, all filters are successfully passed and the deserialization takes place.

143. Monitoring deserialization via JFR

Java Flight Recorder (JFR) is an event-based tool for the diagnosis and profiling of Java applications. This tool was initially added in JDK 7 and, since then, it has been constantly improved. For instance, in JDK 14, JFR was enriched with event streaming, and in JDK 19, with filtering event capabilities, and so on. You can find all JFR events listed and documented per JDK version at <https://sap.github.io/SapMachine/jfrevents/>.

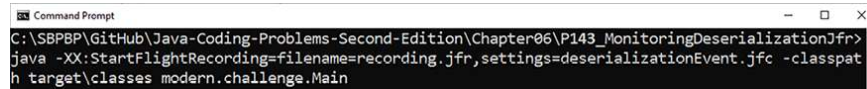
Among its rich list of events, JFR can monitor and record deserialization events (the deserialization event). Let's assume a simple application like the one from *Problem 131* (the first problem of this chapter). We start configuring JFR for monitoring the deserialization of this application by adding `deserializationEvent.jfc` into the root folder of the application:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration version="2.0" description="test">
  <event name="jdk.Deserialization">
    <setting name="enabled">true</setting>
    <setting name="stackTrace">>false</setting>
  </event>
</configuration>
```

Practically, this file instructs JFR to monitor and record the deserialization events.

Next, we use `-XX:StartFlightRecording=filename=recording.jfr` to instruct JFR to record output in a file named `recording.jfr`, and we continue with `settings=deserializationEvent.jfc` to point out the configuration file listed previously.

So, the final command is the one from this figure:

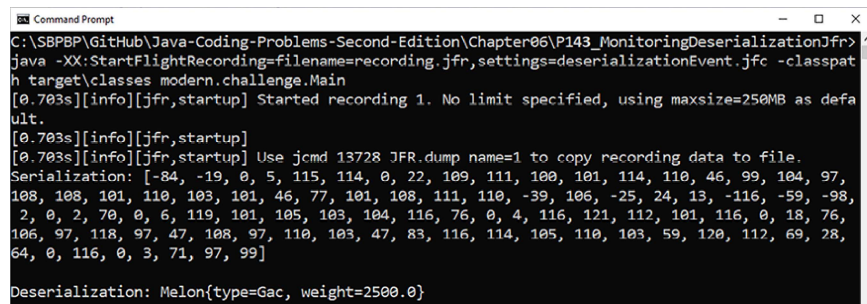


```

C:\SBPBP\GitHub\Java-Coding-Problems-Second-Edition\Chapter06\P143_MonitoringDeserializationJfr>
java -XX:StartFlightRecording=filename=recording.jfr,settings=deserializationEvent.jfc -classpath
target\classes modern.challenge.Main
  
```

Figure 6.2: Running JFR

After this command is executed, you'll see an output as in Figure 6.3:

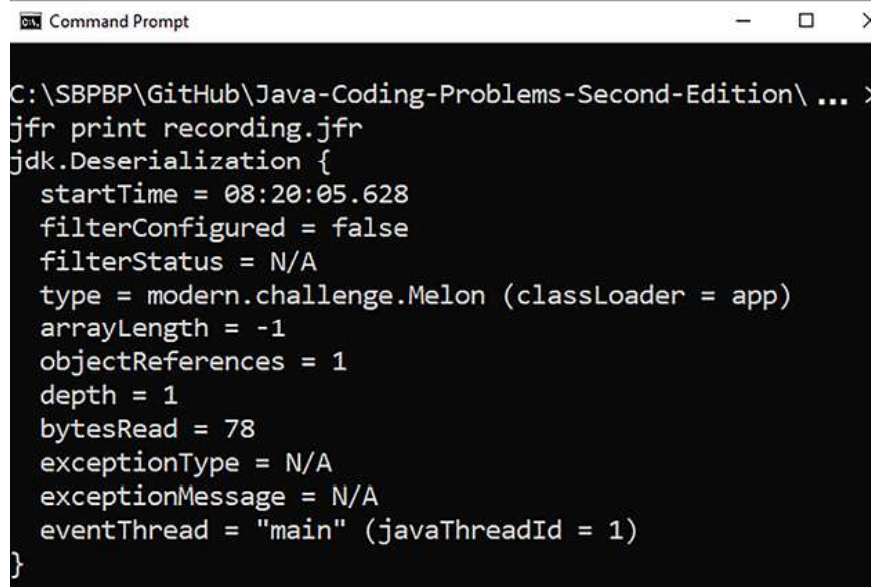


```

C:\SBPBP\GitHub\Java-Coding-Problems-Second-Edition\Chapter06\P143_MonitoringDeserializationJfr>
java -XX:StartFlightRecording=filename=recording.jfr,settings=deserializationEvent.jfc -classpath
target\classes modern.challenge.Main
[0.703s][info][jfr,startup] Started recording 1. No limit specified, using maxsize=250MB as default.
[0.703s][info][jfr,startup]
[0.703s][info][jfr,startup] Use jcmd 13728 JFR.dump name=1 to copy recording data to file.
Serialization: [-84, -19, 0, 5, 115, 114, 0, 22, 109, 111, 100, 101, 114, 110, 46, 99, 104, 97,
108, 108, 101, 110, 103, 101, 46, 77, 101, 108, 111, 110, -39, 106, -25, 24, 13, -116, -59, -98,
2, 0, 2, 70, 0, 6, 119, 101, 105, 103, 104, 116, 76, 0, 4, 116, 121, 112, 101, 116, 0, 18, 76,
106, 97, 118, 97, 47, 108, 97, 110, 103, 47, 83, 116, 114, 105, 110, 103, 59, 120, 112, 69, 28,
64, 0, 116, 0, 3, 71, 97, 99]
Deserialization: Melon{type=Gac, weight=2500.0}
  
```

Figure 6.3: The output of our application

JFR has produced a file named `recording.jfr`. We can easily view the content of this file via the JFR CLI. The command (`jfr print recording.jfr`) and the output are available in Figure 6.4:



```
Command Prompt
C:\SBPBP\GitHub\Java-Coding-Problems-Second-Edition\ ... >
jfr print recording.jfr
jdk.Deserialization {
  startTime = 08:20:05.628
  filterConfigured = false
  filterStatus = N/A
  type = modern.challenge.Melon (classLoader = app)
  arrayLength = -1
  objectReferences = 1
  depth = 1
  bytesRead = 78
  exceptionType = N/A
  exceptionMessage = N/A
  eventThread = "main" (javaThreadId = 1)
}
```

Figure 6.4: The output of JFR containing deserialization information

JFR produced a single deserialization event since our application has performed a single serialization/deserialization cycle of a `Melon` object. You can see the type of the object (here, `Melon`) via the `type` field. Since the `Melon` instance is not an array, the `arrayLength` was set to `-1`, which means that arrays are not applicable. The `objectReferences` represents the first object reference in the stream (so, `1`) and the `bytesRead` represents the number of bytes read from this stream (in this case, `78` bytes). We also see that there was no filter present, `filterConfigured = false`, and `filterStatus = N/A` (not applicable). Moreover, `exceptionType` and `exceptionMessage` are `N/A`. They are not applicable because there is no filter present. They are useful for capturing any exceptions caused by a potential filter.

Besides the JFR CLI, you can use more powerful tools for consuming the deserialization event such as JDK Mission Control

(<https://www.oracle.com/java/technologies/jdk-mission-control.html>) and

the well-known Advanced Management Console

(<https://www.oracle.com/java/technologies/advancedmanagementconsole.html>).

Summary

In this chapter, we covered a bunch of problems dedicated to handling Java serialization/deserialization processes. We started with classical problems and moved on to cover JDK 17 context-specific deserialization filters passing through JDK 9 deserialization filters on the way.

Join our community on Discord

Join our community's Discord space for discussions with the author and other readers:

<https://discord.gg/8mgytp5DGQ>

